Evaluating Renewable Energy System Reliability with Monte Carlo Simulation and Inference

By Romand Lansangan

Summative Assessment 1 in Statistical Computing

Daily Energy Demand

On a daily basis, energy demand depends on household activity patterns:

- 60% of days are regular $ \to D_t \sim N(70,4^2)$
- 30% are hot days with more appliance usage $ \to D_t \sim N(80,5^2)$
- 10% are festive days $ \to D_t \sim N(90,6^2)$

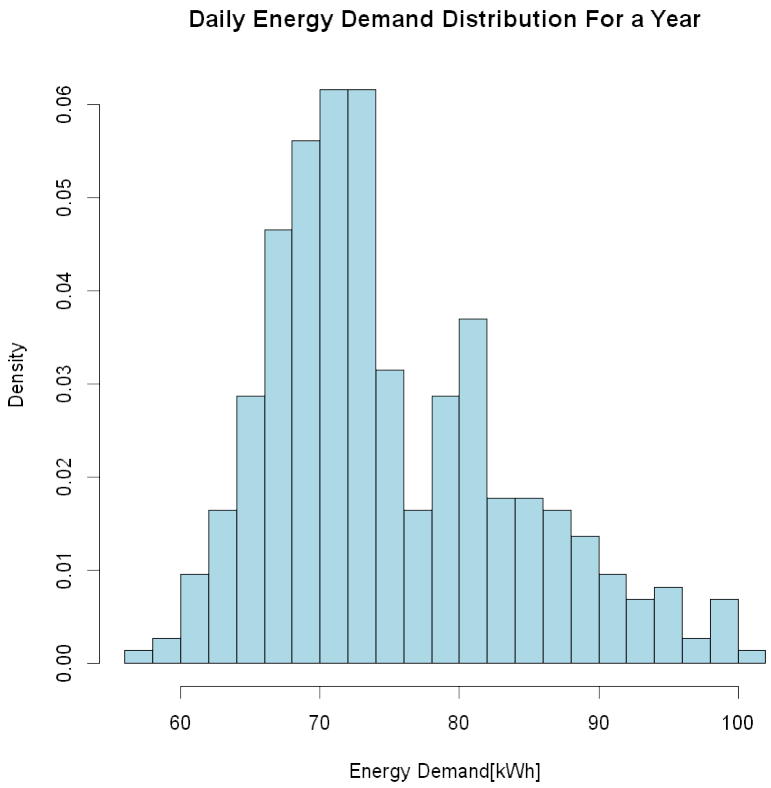
Simulate daily energy demand Dt using this mixture.

```
library(tidyverse)
generate_demand <- function() {
  days <- 365
  d1 <- rnorm(days, mean = 70, sd = 4)
  d2 <- rnorm(days, mean = 80, sd = 5)
  d3 <- rnorm(days, mean = 90, sd = 6)
  dt <- matrix(c(d1,d2,d3),nrow = days, ncol=3)

  pr <- c(0.60,0.30,0.10)
  demand_cat <- sample(1:3, days, replace = TRUE, prob = pr)

  demand <- numeric(days)
  for(day in 1:days) {
    demand[day] <- dt[day, demand_cat[day]]
  }
  return (demand)
}
demand <- generate_demand()
```

```
-- Attaching core tidyverse packages ------------------------
tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.0
v ggplot2   3.4.2     v tibble    3.2.1
v lubridate 1.9.3     v tidyr     1.3.1
v purrr     1.0.2
-- Conflicts ------------------------------------------
tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force
all conflicts to become errors
```

```
hist(demand, breaks = 30, col = "lightblue",
     main = "Daily Energy Demand Distribution For a Year",
     xlab = "Energy Demand[kWh]", probability = TRUE)
```




```
cat("Demand mean:", mean(demand), "\nDemand Var:", var(demand))
```

```
Demand mean: 75.02809
Demand Var: 76.70079
```

Based on one simulation of the daily energy demand, the daily mean is around 75.29965 and variance with 62.09065. Which make sense since:

$$E[\text{daily demand}] = 0.60\mu_1 + 0.30\mu_2 + 0.10\mu_3 = 0.60*70 + 0.30*80 + 0.10*90$$

```
0.60*70 + 0.30 *80 + 0.10*90
```

```
[1] 75
```

So it's close to the expected demand.

Also the histogram showed a slightly skewed but approximately normal distribution. As we can see most, around 60%, of the data fell around 70 (which is d1) and there's a slight bump around 80 (which is d2) and finally the skewness that's been caused by d3.

There are of course few flaws to this model:

- Although the distribution seemed to be rightly skewed, real demand can have (rare) extreme spikes than that of normal distribution. For that reason, we can mix in other distribution like t-distribution or log-normal distribution for such extreme events. Since we are using mixture, we need not to assume identical distribution, just independent.
- Also, the model assumed a discrete mixture when in fact it should be continous mixture since temperature affects demand continously. We can't just jump to another distribution just cause we specifically reached a certain threshold. The way to get around this is to add a distribution for temperature propose a specific type of relationship to the demand(for example quadratic) then factor that to the base demand.
- Also, this does not take into account the trend of demand. Anual demand rarely stays constant due to changes in population or technological advancement and many more factors. To solve this we can a trend component to the model.

While these limitations could be addressed with more complex models (continuous mixtures, heavy-tailed components, trend modeling), we proceed with the current discrete mixture model for several reasons:

- It provides a reasonable first-order approximation
- The added complexity may not justify the improvement given data constraints
- The discrete categories (regular/hot/festive) are interpretable for stakeholders

Energy Production

Suppose energy from individual solar bursts (e.g., sunlight intervals with enough intensity) can be modeled as a Poisson process.

- The number of bursts per day, N_t, follows a Poisson distribution with mean $\lambda = 10$.
- Each burst contributes a random energy amount $E_i \sim Exp\left(\beta = \frac{1}{3}\right) kWh$

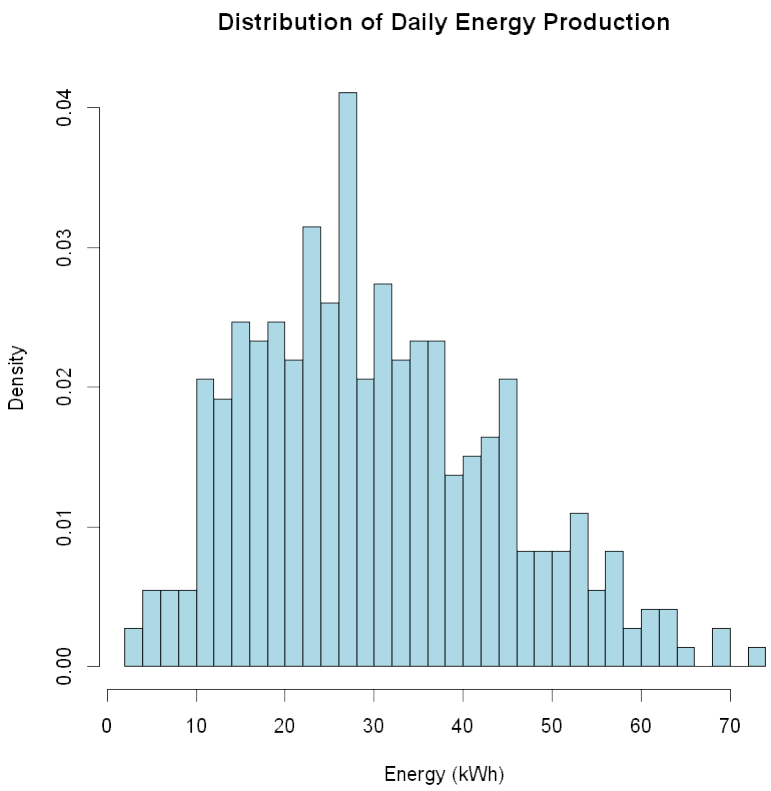
Define daily energy production as:

$$Y_t = \sum_{i=1}^{N_t} E_i$$

Simulate for 365 days

```
production_simulate <- function(){
  lambda <- 10
  beta <- 1/3
  days <- 365

  # intialize vector for daily energy prod.
```

```r
  production <- numeric(days)

  # Simulate each day
  for(day in 1:days) {
    # Number of bursts for this day
    N_t <- rpois(1, lambda)

    # Energy from each burst for the day
    if(N_t > 0) {
      E_i <- rexp(N_t, rate = beta)
      production[day] <- sum(E_i)
    } else {
      production[day] <- 0
    }
  }
  return(production)
}
production <- production_simulate()

hist(production, breaks = 30, col = "lightblue",
     main = "Distribution of Daily Energy Production",
     xlab = "Energy (kWh)", freq = FALSE)
```

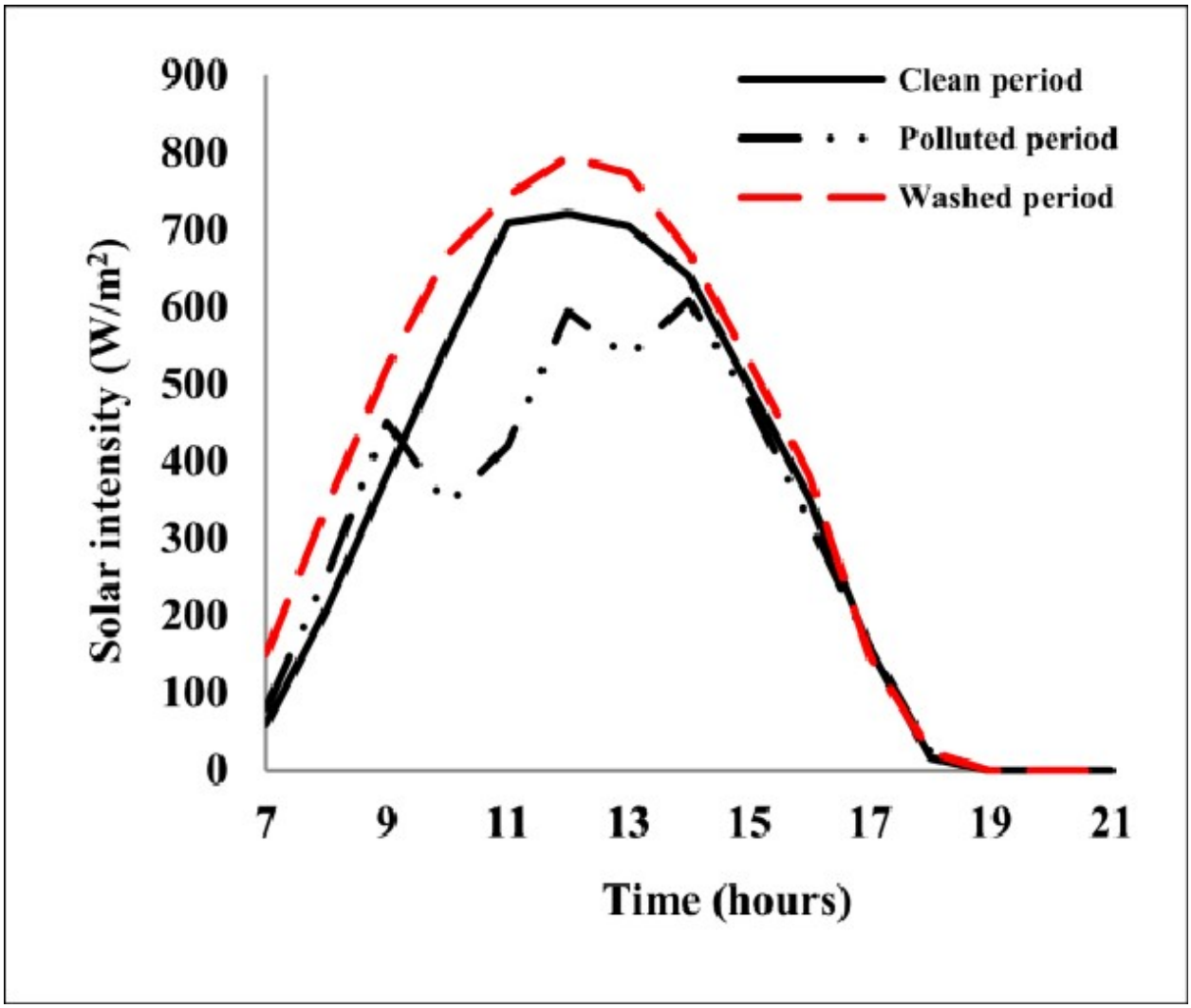


```
cat("Energy Production Mean:", mean(production), "\nEnergy Production
Var:", var(production))
```

```
Energy Production Mean: 29.99176
Energy Production Var: 191.5604
```

There are also many flaws to the enegy production model much like the demand model:

- Seasonal patterns was disregarded. It's unrealistic to assume the during winter or summer, the $\lambda = 10$ will stay the same.
- There might be multi-correlation involve since whether patterns too creates one. The current model treats each day independently.
- Solar intensity follows a more normal distribution throughout the day (see graph below).



Chaichan, Miqdam & Abass, Khaleel & Kazem, Hussein A. (2018). Dust and Pollution Deposition Impact on a Solar Chimney Performance. International Research Journal of Advanced Engineering and Science. 3. 127-132.

link here

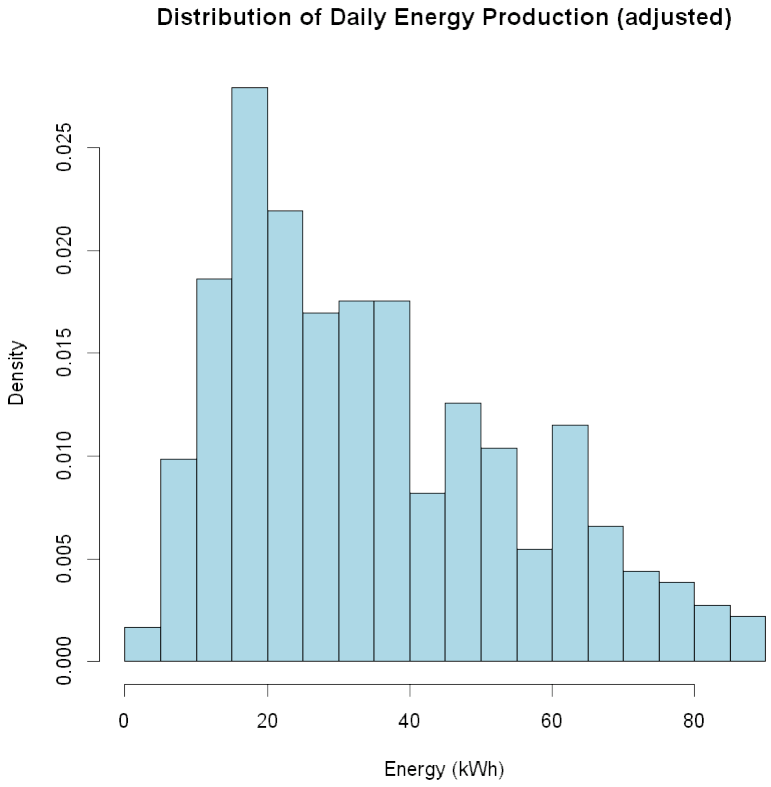
For the sake of the model, we'll incorporate the 1st and 3rd points to the new model.

```
production_simple_seasonal <- function() {
  days <- 365
  production <- numeric(days)

  for(day in 1:days) {
    # Seasonal adjustment for lambdal
    # incorporated from fourier series
    seasonal_factor <- 1 + 0.3 * sin(2 * pi * (day - 80) / 365)

    lambda_day <- 10 * seasonal_factor

    N_t <- rpois(1, lambda_day)


    if(N_t > 0) {
      # Gamma instead of Exponential
      # Shape > 1 creates bell-shaped curve like solar intensity
      E_i <- rgamma(N_t, shape = 2, rate = 0.6)
      production[day] <- sum(E_i) * seasonal_factor

    }
  }

  return(production)
}
```

IN the changes we incorporate first Fourier series (good for modeling seasons) to add some seasonal component (both λ and overall production were affected) and change Exponential distribution to Gamma distribution to better estimate given the solar intensity distribution.

```
production <- production_simple_seasonal()

hist(production, breaks = 30, col = "lightblue",
     main = "Distribution of Daily Energy Production (adjusted)",
     xlab = "Energy (kWh)", freq = FALSE)
cat("Energy Production Mean:", mean(production), "\nEnergy Production
Var:", var(production))
```

```
Energy Production Mean: 34.96576
Energy Production Var: 406.1374
```



Monte Carlo Estimation

Use Monte Carlo to estimate the annual expected energy output. Determine the variance and standard error of the estimate.

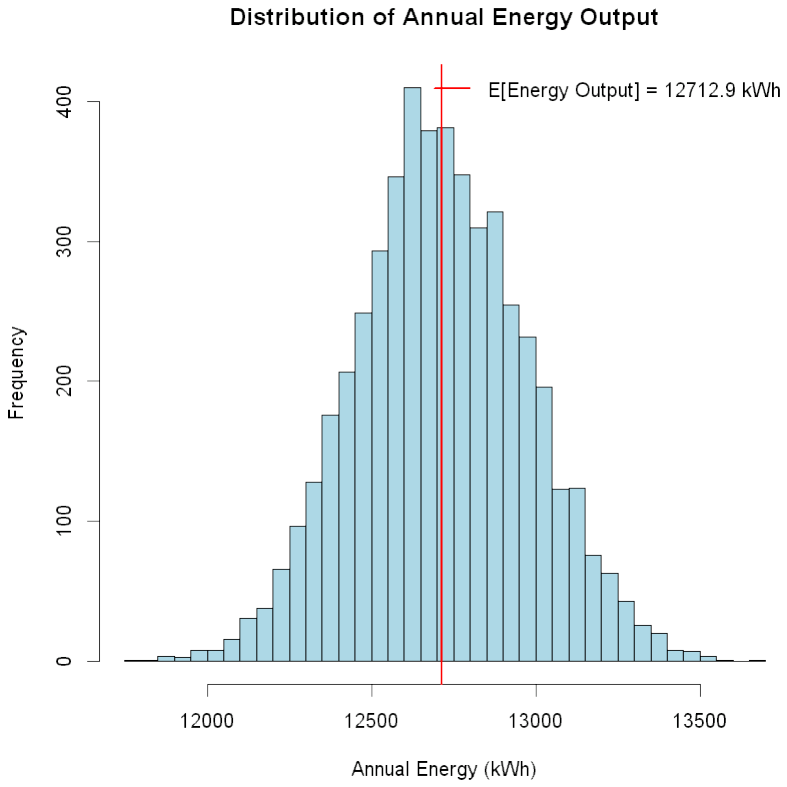
For a more efficient code, let us vectorized some of the generation instead of generating it one by one.

```
production_vectorized <- function() {
  days <- 365

  # Pre-compute seasonal factors
  seasonal_factors <- 1 + 0.3 * sin(2 * pi * ((1:days) - 80) / 365)
  lambda_days <- 10 * seasonal_factors

  # Generate all N_t values at once
  N_t <- rpois(days, lambda_days)

  # Generate all bursts at once
  total_bursts <- sum(N_t)
  if(total_bursts > 0) {
    E_i <- rgamma(total_bursts, shape = 2, rate = 0.6)

    # Map bursts back to days
```

```
    production <- numeric(days)
    burst_idx <- 1
    for(day in 1:days) {
      if(N_t[day] > 0) {
        day_bursts <- E_i[burst_idx:(burst_idx + N_t[day] - 1)]
        production[day] <- sum(day_bursts) * seasonal_factors[day]
        burst_idx <- burst_idx + N_t[day]
      }
    }
    return(sum(production))
  } else {
    return(0)
  }
}
# function call for simple mc
simple_mc <- function(n_sim){
    return(replicate(n_sim, production_vectorized()))
}

# Run Monte Carlo simulation
n_sim <- 5000
simple_mc_res <- simple_mc(n_sim)

expected_output <- mean(simple_mc_res)
variance_output <- var(simple_mc_res)
std_error <- sd(simple_mc_res) / sqrt(n_sim)

cat("Expected Anual Energy Output (estimation):", expected_output,
"kWh\n")
cat("Variance:", variance_output, "\n")
cat("Std Error:", std_error, "\n")
```

```
Expected Anual Energy Output (estimation): 12712.88 kWh
Variance: 68434.5
Std Error: 3.699581
```

```
hist(simple_mc_res, breaks = 50, col = "lightblue",
     main = "Distribution of Annual Energy Output",
     xlab = "Annual Energy (kWh)")
abline(v = expected_output, col = "red", lwd = 2)

legend("topright",
       legend = c(paste("E[Energy Output] =", round(expected_output,
1), "kWh")),
       col = "red",
       lwd = 2,
       bty = "n")  # No box around legend
```



As we can see, the distribution closely resembles a normal distribution. This is due to the Central Limit Theorem (CLT), as we are summing 365 independent daily production values in each simulation. Even though daily productions have different distributions across seasons (non-identical), the CLT still applies because we have a large number of summands (365 days) with finite variance.

Improve Variance

Antithetic

```
production_antithetic <- function() {
    days <- 365
    seasonal_factors <- 1 + 0.3 * sin(2 * pi * ((1:days) - 80) / 365)
    lambda_days <- 10 * seasonal_factors

   # generate both paths' burst counts
    U_pois <- runif(days)
    N_t1 <- qpois(U_pois, lambda_days)
    N_t2 <- qpois(1 - U_pois, lambda_days)

    # generate shared uniforms for gamma;
    # needed for negative correlation between prod1 and prod2
    max_bursts <- max(sum(N_t1), sum(N_t2))
    U_gamma <- runif(max_bursts)
```

```
    # original path
    total_bursts1 <- sum(N_t1)
    prod1 <- 0
    if(total_bursts1 > 0) {
        E1 <- qgamma(U_gamma[1:total_bursts1], shape = 2, rate = 0.6)
        idx <- 1
        for(day in 1:days) {
            if(N_t1[day] > 0) {
                prod1 <- prod1 + sum(E1[idx:(idx + N_t1[day] - 1)]) *
seasonal_factors[day]
                idx <- idx + N_t1[day]
            }
        }
    }

    # antithetic path
    total_bursts2 <- sum(N_t2)
    prod2 <- 0
    if(total_bursts2 > 0) {
        E2 <- qgamma(1 - U_gamma[1:total_bursts2], shape = 2, rate =
0.6)
        idx <- 1
        for(day in 1:days) {
            if(N_t2[day] > 0) {
                prod2 <- prod2 + sum(E2[idx:(idx + N_t2[day] - 1)]) *
seasonal_factors[day]
                idx <- idx + N_t2[day]
            }
        }
    }

    return((prod1 + prod2) / 2)
}


production_antithetic_mc <- function(n_sim){
    return(replicate(n_sim, production_antithetic()))
}
```

Control Variates

```
production_control_variate_mc <- function(n_sim) {
    days <- 365
    seasonal <- 1 + 0.3 * sin(2 * pi * ((1:days) - 80) / 365)

    prod_with_control <- function() {
        N_t <- rpois(days, 10 * seasonal)
        prod <- 0
```

```
        # total energy (not just bursts)
        total_energy <- 0

        for(day in 1:days) {
            if(N_t[day] > 0) {
                E <- rgamma(N_t[day], shape = 2, rate = 0.6)
                daily_energy <- sum(E)
                prod <- prod + daily_energy * seasonal[day]
                total_energy <- total_energy + daily_energy
            }
        }
        c(prod, total_energy)
    }

    # theorethical expected total energy
    expected_bursts_per_day <- 10
    expected_energy_per_burst <- 2/0.6  # E[Gamma(2, 0.6)]
    expected_total_energy <- days * expected_bursts_per_day *
expected_energy_per_burst

    # estimate best c
    prelim <- replicate(200, prod_with_control())
    c_star <- -cov(prelim[1,], prelim[2,]) / var(prelim[2,])

    # main simulation
    results <- replicate(n_sim, prod_with_control())
    productions <- results[1,]
    controls <- results[2,]

    # application of control variate
    productions + c_star * (controls - expected_total_energy)
}
```

Stratified

```
production_stratified <- function(k) {
    days <- 365
    seasonal_factors <- 1 + 0.3 * sin(2 * pi * ((1:days) - 80) / 365)

    total_production <- 0

    for(day in 1:days) {
        # apply seasonal to base rate
        lambda <- 10 * seasonal_factors[day]

        # randomly select which stratum we're in
        stratum <- sample(1:k, 1)
```

```
        # generate uniform random variable within the stratum bounds
        u <- runif(1, min = (stratum - 1) / k, max = stratum / k)
        N_t <- qpois(u, lambda)

        # generate production amounts if we have items
        if(N_t > 0) {
            E <- rgamma(N_t, shape = 2, rate = 0.6)
            total_production <- total_production + sum(E)
        }
    }

    return(total_production)
}

production_stratified_mc <- function(n_sim, k = 4) {
    return(replicate(n_sim, production_stratified(k)))
}
```

Note that in stratified, we'll define $k=4$ since there are 4 seasons in a year (it captures the 4 phases of the sine waves).

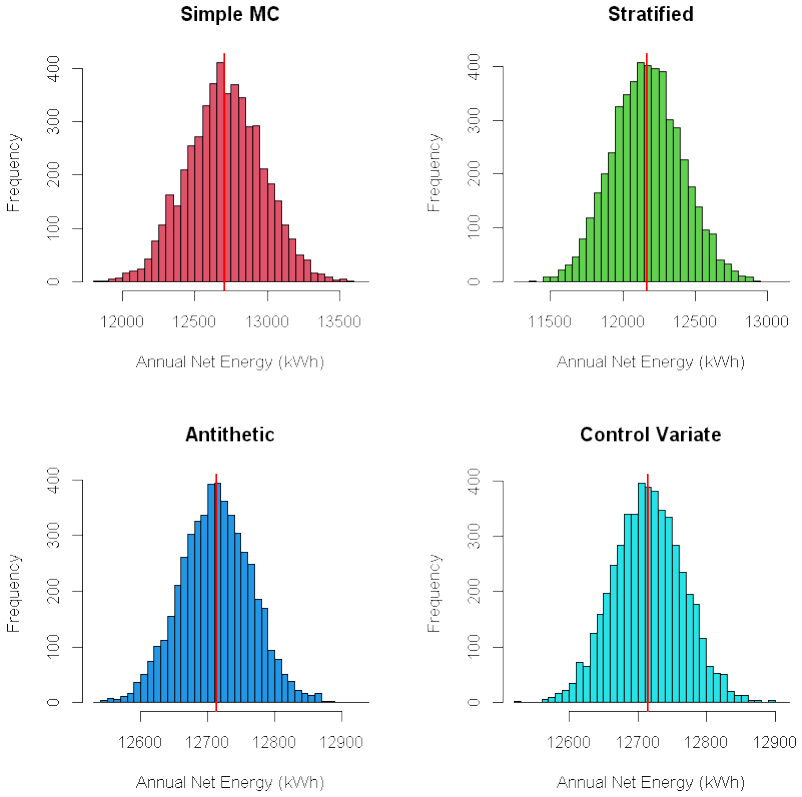
Simulate ALL

```
n_sim <- 5000

t1 <- system.time(res_simple <- simple_mc(n_sim))
t2 <- system.time(res_stratified <- production_stratified_mc(n_sim))
t3 <- system.time(res_antithetic <- production_antithetic_mc(n_sim))
t4 <- system.time(res_control <- production_control_variate_mc(n_sim))

methods <- list(
  "Simple MC" = res_simple,
  "Stratified" = res_stratified,
  "Antithetic" = res_antithetic,
  "Control Variate" = res_control
)

cat(sprintf("%-20s %10s %10s %10s %12s\n", "Method", "Mean",
"Variance", "Time(s)", "Var Reduction"))
cat(paste(rep("=", 65), collapse = ""), "\n")

times <- c(t1[3], t2[3], t3[3], t4[3])
base_var <- var(methods[[1]])  # Simple MC is baseline

for(i in 1:length(methods)) {
  var_reduction <- if(i == 1) {
    "-"  # Baseline has no reduction
  } else {
    sprintf("%.1f%%", (1 - var(methods[[i]])/base_var) * 100)
  }
```

```
  cat(sprintf("%-20s %10.2f %10.2f %10.3f %12s\n",
              names(methods)[i],
              mean(methods[[i]]),
              var(methods[[i]]),
              times[i],
              var_reduction))
}
```

```
Method                       Mean   Variance    Time(s) Var Reduction
================================================================

Simple MC                12708.15   69263.88      4.750            -
Stratified               12167.34   59242.50     38.110        14.5%
Antithetic               12712.72    2778.10     55.250        96.0%
Control Variate          12714.98    2599.85     12.450        96.2%
```

```
# Visualization
par(mfrow = c(2, 2))

# Histograms
for(i in 1:4) {
  hist(methods[[i]], breaks = 30, col = i+1,
       main = names(methods)[i],
       xlab = "Annual Net Energy (kWh)")
  abline(v = mean(methods[[i]]), col = "red", lwd = 2)
}

par(mfrow = c(1, 1))
```



Based on the metrics above, all have achieved simmilar result in terms of expected anual production but Antithetic is the clear winner. Although it's the slowest among the 4 method, it seperates itself from the rest in terms of Variance Reduction with around 96%. It indeed seperate itself since it's the only the achieves its goal of reducing variance.

- For stratify, most variance comes from the random burst counts and energy values, not from predictable seasonal patterns so the main variance cannot be stratified.
- For antithetic, remember that we have used gamma and poisson distrubution through the generation of $U \sim Unif(0,1)$. In both distribution, gamma and poisson, higher U means higher result--burst count in qpois and energy values in qgamma. This strongly negatively correlated production estimates between the path using U and path using $1-U$ (which was also amplified by integrating of seasonal factors to the model), by around -0.92 as per var reduction. Although the biggest drawback is its runtime with almost a minute.
- For control variate, it works as well as antithetic while running for only 10% of its runtime.

Why Does Control Variate work so well?

Some reason might be:

- Known Expectation - we could calculate for theoretical expectation. On most real world scenario, this would not be possible. But in this model, it's possible.

- Strong correlation with total energy - since we used the total daily energy (sum of all burst) instead of just number of burst, we have captured a strong correlation with the anual production.
- Linear despite seasonal complexity added - although we added a seasonal component that intruces non linearity, it appears that the total energy and daily production remains linear.
- Computational efficiency - unlike antithetic, we only need one simulation path.

So taking into all of that, the optimal choice is Control Variate.

Let us now create function but instead of returning anual, we'll return daily.

```
production_control_daily <- function(days) {
    seasonal_factors <- 1 + 0.3 * sin(2 * pi * ((1:days) - 80) / 365)
    lambda_days <- 10 * seasonal_factors

    # generate burst counts
    N_t <- rpois(days, lambda_days)

    daily_prod <- numeric(days)
    total_energy <- 0  # control variable

    total_bursts <- sum(N_t)
    if(total_bursts > 0) {
        # all burst energies at once
        E_all <- rgamma(total_bursts, shape = 2, rate = 0.6)
        total_energy <- sum(E_all)  # control variable

        # assign energies to days
        idx <- 1
        for(day in 1:days) {
            if(N_t[day] > 0) {
                day_energies <- E_all[idx:(idx + N_t[day] - 1)]
                daily_prod[day] <- sum(day_energies) *
seasonal_factors[day]
                idx <- idx + N_t[day]
            }
        }
    }

    total_prod <- sum(daily_prod)

    return(list(
        "daily_prod" = daily_prod,
        "daily_burst" = N_t,
        "total_prod" = total_prod,
        "total_energy" = total_energy
    ))
}
```

Failures

Assume the microgrid fails on day t if $Y_t < D_t$. Estimate:

$$P(\text{at least 10 failures in a year})$$

Use Monte Carlo simulation to estimate this probability and construct a 95% confidence interval for the true probability.

```r
# for reminder
generate_demand <- function(days) {
    d1 <- rnorm(days, mean = 70, sd = 4)
    d2 <- rnorm(days, mean = 80, sd = 5)
    d3 <- rnorm(days, mean = 90, sd = 6)
    dt <- matrix(c(d1,d2,d3),nrow = days, ncol=3)

    pr <- c(0.60,0.30,0.10)
    demand_cat <- sample(1:3, days, replace = TRUE, prob = pr)

    demand <- numeric(days)
    for(day in 1:days) {
    demand[day] <- dt[day, demand_cat[day]]
    }
    return (demand)
}

failure_control_variate <- function() {
    days <- 365
    demand <- generate_demand(days)

    # generate production with control
    prod_result <- production_control_daily(days)
    daily_prod <- prod_result$daily_prod
    total_energy <- prod_result$total_energy

    failures <- sum(daily_prod < demand)
    indicator <- as.numeric(failures >= 10)

    return(c(indicator, total_energy))
}

# Bootstrap resampling for validation
failure_probability_control_mc <- function(n_sim, n_bootstrap = 1000)
{
    # expected control value
    expected_total_energy <- 365 * 10 * (2/0.6)

    # prelimenary run for c_star
    prelim <- replicate(200, failure_control_variate())
    c_star <- -cov(prelim[1,], prelim[2,]) / var(prelim[2,])
```

```
    # main simulation
    results <- replicate(n_sim, failure_control_variate())
    indicators <- results[1,]
    controls <- results[2,]

    # application of control variate
    adjusted <- indicators + c_star * (controls -
expected_total_energy)
    adjusted <- pmax(0, pmin(1, adjusted))  # Keep in [0,1]

    p_hat <- mean(adjusted)

    # Bootstrap
    bootstrap_means <- replicate(n_bootstrap, {
        mean(sample(adjusted, n_sim, replace = TRUE))
    })

    return(list(
        probability = p_hat,
        se = sd(bootstrap_means),
        ci_95 = quantile(bootstrap_means, c(0.025, 0.975)),
        c_star = c_star
    ))
}

n_sim <- 5000
failures <- failure_probability_control_mc(n_sim)

cat("Failure Probability Estimation\n")
cat("==============================\n")
cat(sprintf("MC with Control Variate:  \nP = %.4f, Var = %.4f\n",
failures$probability, failures$se))
cat(sprintf("95%% CI: [%.4f, %.4f]", failures$ci_95[1],
failures$ci_95[2]))
```

```
Failure Probability Estimation
==============================
MC with Control Variate:
P = 1.0000, Var = 0.0000
95% CI: [1.0000, 1.0000]
```

Based on the result above, it becomes clear the system that the solar-powered microgrid have in place is bound to fail. This simply means that the microgrid is not producing enough energy to satisfy the given demand.

Remember that

$$E[\text{daily demand}] = 0.60\mu_1 + 0.30\mu_2 + 0.10\mu_3 = 0.60*70 + 0.30*80 + 0.10*90 = 75\,kWh$$

Given the result the Monte Carlo with Antithetic, the expected daily production should be:

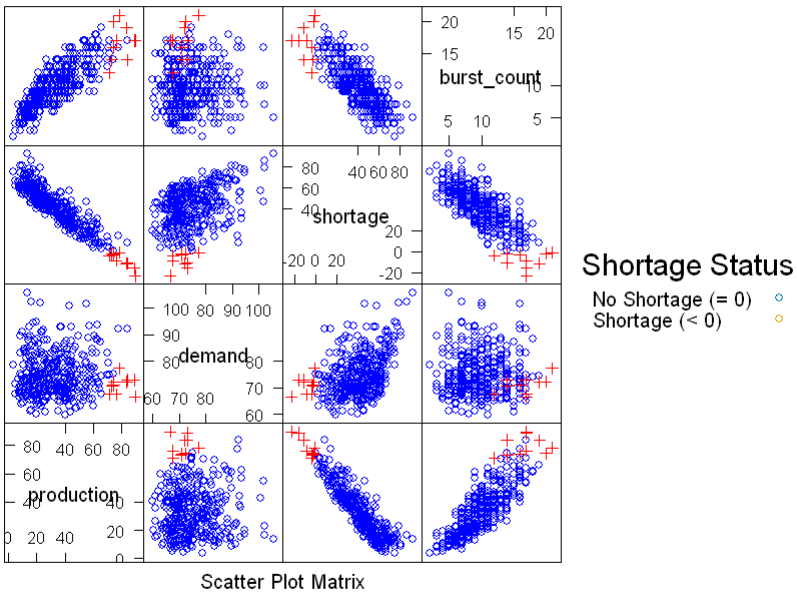
```r
expected_prod <- mean(res_control)/365
expected_deman <- 75
cat("E[daily prod] =", expected_prod, "\n")
cat("Produces:", expected_prod/ expected_deman)
```

```
E[daily prod] = 34.83556
Produces: 0.4644741
```

This is less than half of the demand, only 46% of what should be produced was being produced! So it's no wonder the system is bound to fail.

What to do with this information?

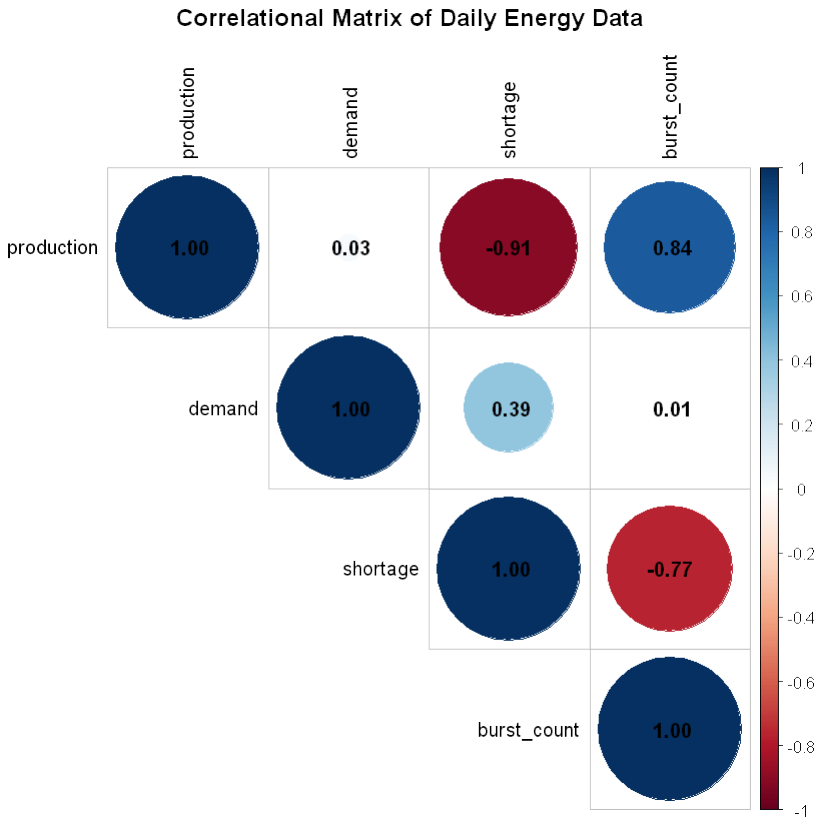
```r
days <- 365
production <- production_control_daily(days)

demand <- generate_demand(days)

energy_data <- data.frame(
  day = 1:days,
  production = production$daily_prod,
  demand = demand,
  shortage = demand - production$daily_prod,
  burst_count = production$daily_burst
)

library(lattice)
splom(~energy_data[2:5],
      groups = energy_data$shortage < 0,
      pch = c(1, 3),
      col = c("blue", "red"),
      auto.key = list(
        space = "right",
        title = "Shortage Status",
        text = c("No Shortage (≥ 0)", "Shortage (< 0)")
      ))
```



Scatter Plot Matrix

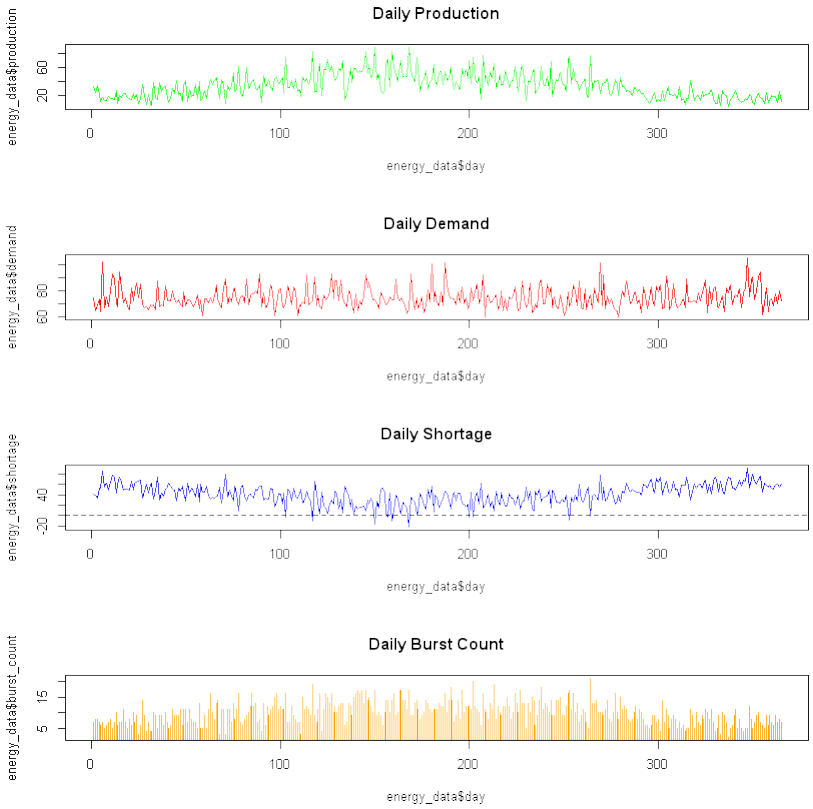
```
library(corrplot)
corrMat <- cor(energy_data[,2:5])
corrplot(corrMat, method = "circle", type = "upper",
         addCoef.col = "black", tl.col = "black",
         title = "Correlational Matrix of Daily Energy Data",
         mar = c(0, 0, 2, 0))
```

```
corrplot 0.95 loaded
```



The scatter plot matrix reveals the fundamental issue: production and demand are essentially independent (-0.03 correlation), meaning the system cannot adapt production to meet demand. The strong negative correlation between production and shortage (-0.91) and positive correlation between demand and shortage (0.39) confirm that failures are driven primarily by insufficient production rather than demand spikes.

```
par(mfrow = c(4,1))
plot(energy_data$day, energy_data$production, type = "l",
     col = "green", main = "Daily Production")
plot(energy_data$day, energy_data$demand, type = "l",
     col = "red", main = "Daily Demand")
plot(energy_data$day, energy_data$shortage, type = "l",
     col = ifelse(energy_data$shortage < 0, "red", "blue"),
     main = "Daily Shortage")
abline(h = 0, lty = 2)
plot(energy_data$day, energy_data$burst_count, type = "h",
     col = "orange", main = "Daily Burst Count")
```



The time series visualization clearly shows:

- Production exhibits strong seasonal variation (sine wave pattern due to our model)
- Demand remains relatively stable with occasional spikes
- Shortage is persistent throughout the year, with blue regions (shortage < 0) being rare. However there is a season between 100 to 200 that signifies more production. This might indicate the summer time or the year.
- Burst count follows the seasonal pattern, peaking mid-year, in line with reduction of shortage and production increase.

This temporal mismatch is critical - even when production peaks in summer, it barely meets demand, while winter months face severe shortages.

```r
library(plotly)

plot_ly(energy_data,
        x = ~production,
        y = ~demand,
        z = ~shortage,
        color = ~(burst_count > 10),
        colors = c("blue", "red"),
        type = "scatter3d",
        mode = "markers",
        marker = list(size = 5)) %>%
```

```
  layout(title = "Production vs Demand vs Shortage",
         scene = list(xaxis = list(title = "Production"),
                      yaxis = list(title = "Demand"),
                      zaxis = list(title = "Shortage")));
```


```
{"attrs":{"58f47eb425d0":{"alpha_stroke":1,"color":{},"colors":
["blue","red"],"marker":{"size":5},"mode":"markers","sizes":
[10,100],"spans":[1,20],"type":"scatter3d","x":{},"y":{},"z":
{}}},"base_url":"https://plot.ly","config":{"modeBarButtonsToAdd":
["hoverclosest","hovercompare"],"showSendToCloud":false},"cur_data":"5
8f47eb425d0","data":[{"error_x":{"color":"rgba(0,0,255,1)"},"error_y":
{"color":"rgba(0,0,255,1)"},"line":
{"color":"rgba(0,0,255,1)"},"marker":
{"color":"rgba(0,0,255,1)","line":
{"color":"rgba(0,0,255,1)"},"size":5},"mode":"markers","name":"FALSE",
"textfont":{"color":"rgba(0,0,255,1)"},"type":"scatter3d","x":
[17.3489,17.4888,21.5385,11.5136,20.8125,15.2263,19.2961,15.9279,9.888
1,18.5791,16.1731,21.7003,14.8523,15.7694,23.3081,18.1538,20.9022,21.4
84,11.3137,21.0566,15.9145,13.6686,21.2165,19.2015,20.4781,17.6653,16.
1058,13.5405,17.8767,18.4395,23.6119,20.793,22.6098,16.6594,24.9565,17
.5596,17.9638,25.0251,29.1231,19.4607,21.2898,23.5867,22.8698,17.7911,
27.4111,29.9173,22.6077,23.9993,22.4353,19.903,17.2608,26.2649,22.6422
,29.5814,32.9893,24.7781,30.1167,30.8749,15.7723,31.1163,33.2117,28.03
33,29.2435,25.1728,25.9605,38.3904,34.149,25.0826,27.199,26.5771,25.22
07,30.6684,25.9206,30.0225,39.7693,31.3643,37.7301,35.1033,41.4822,34.
867,23.4765,41.0526,37.146,33.0488,41.6974,29.9485,37.3366,26.4504,37.
0526,37.3425,29.4634,35.3877,26.4594,29.9609,34.6683,30.9746,31.3229,2
6.126,34.8605,33.8843,34.8126,19.6931,39.3185,37.9805,21.4813,25.084,2
8.7457,27.3134,41.5396,26.9004,16.9397,23.4418,27.6265,29.9538,22.4007
,19.2961,22.7119,21.9739,33.5528,27.619,21.7307,27.1175,24.0435,21.812
8,31.5612,28.87,23.9513,17.6109,27.552,18.7532,20.3173,19.9021,18.4185
,27.8376,27.7794,15.4302,18.4366,36.0634,17.6749,21.3425,20.1679,16.07
53,19.3759,26.459,23.9054,14.1507,20.8959,19.9785,19.5937,16.1174,14.9
836,15.5381,21.4878,16.3443,16.7922,20.5973,9.5823,18.7205,18.7425,13.
3657,19.6066,11.4016,16.7646,15.2319,18.7773,17.8818,15.6433,12.9732,2
1.0728,20.1371,16.48,19.5995,12.6228,14.9216,17.8589,12.2362,23.3719,1
7.9594,15.2267,24.0743,13.1326,16.829,16.2743,12.8062,18.4063,15.012,2
1.2772,11.578,19.0896,11.2517,14.8264,17.4464,10.5508,15.487],"y":
[71.8743,88.0807,75.2528,94.9321,92.278,72.7707,62.0135,73.4541,66.519
7,70.4247,78.6315,94.6699,73.0573,70.5397,73.0989,72.889,67.8646,73.50
71,87.1095,87.0727,79.1528,75.7578,74.7911,71.4894,67.5386,68.922,70.3
502,79.3414,74.8042,87.8179,94.9614,64.8491,69.9291,72.3081,79.1729,81
.3235,88.1433,74.3985,64.6364,72.2341,79.4687,71.0489,95.9704,84.7361,
73.9714,69.2725,75.115,79.686,77.9621,71.8968,70.61,71.117,72.2367,76.
2085,77.233,86.4499,94.1651,65.3366,71.5279,83.7212,68.3239,88.7567,68
.1036,73.5632,88.5942,84.3088,67.1597,67.2987,68.5324,63.5322,77.4109,
71.6418,80.9498,73.3741,72.762,66.076,78.1442,73.1976,70.0727,81.3695,
68.8777,90.577,84.7192,75.1308,81.7784,66.7633,90.0544,78.8364,87.2024
,74.9808,82.5989,74.2202,76.0631,80.1214,69.6982,78.8872,60.7245,69.14
51,77.4698,78.8756,69.412,70.255,70.5984,68.7748,75.1133,67.9496,68.63
```

```
99,73.2207,68.1913,84.9232,75.6338,61.8055,76.1658,97.6726,91.681,83.0
503,61.3302,86.3218,79.3762,68.974,92.7981,72.1506,78.9842,70.282,79.5
987,79.1321,90.1895,74.7568,71.8206,72.111,80.479,88.9804,76.3728,69.0
263,89.4689,72.3744,83.1501,82.7937,72.6852,71.1152,72.893,86.1689,80.
454,64.2432,98.4462,68.154,74.8937,66.9193,71.199,84.668,70.5912,76.18
36,82.949,68.1609,72.0989,76.6742,82.2729,81.4667,67.038,71.5671,77.43
79,76.9104,74.2419,92.6744,70.4711,77.6476,102.5382,72.7844,72.2046,73
.2,61.7334,66.7405,63.628,71.0447,79.5097,75.5853,66.9879,80.3776,84.2
386,71.3685,71.9113,76.9875,74.3809,76.4044,77.01,74.1043,81.6257,65.1
147,67.849,71.0024,71.2588,83.3548,78.6981,68.477],"z":
[54.5254,70.5919,53.7143,83.4185,71.4655,57.5444,42.7174,57.5262,56.63
17,51.8457,62.4583,72.9696,58.205,54.7703,49.7908,54.7352,46.9624,52.0
231,75.7958,66.0161,63.2383,62.0891,53.5746,52.2878,47.0606,51.2567,54
.2444,65.8009,56.9275,69.3784,71.3494,44.0561,47.3193,55.6488,54.2165,
63.7639,70.1796,49.3734,35.5134,52.7734,58.1789,47.4622,73.1005,66.945
1,46.5602,39.3553,52.5073,55.6867,55.5268,51.9938,53.3492,44.852,49.59
45,46.6271,44.2436,61.6718,64.0484,34.4618,55.7556,52.6048,35.1122,60.
7234,38.8601,48.3903,62.6337,45.9184,33.0107,42.2161,41.3334,36.9551,5
2.1902,40.9735,55.0292,43.3516,32.9927,34.7117,40.414,38.0943,28.5905,
46.5026,45.4012,49.5244,47.5732,42.0821,40.081,36.8148,52.7178,52.3861
,50.1498,37.6384,53.1355,38.8325,49.6037,50.1605,35.0299,47.9127,29.40
16,43.0191,42.6092,44.9913,34.5994,50.562,31.2798,30.7944,53.632,42.86
57,39.8942,45.9073,26.6518,58.0228,58.6941,38.3637,48.5392,67.7188,69.
2803,63.7542,38.6183,64.3479,45.8234,41.355,71.0674,45.0331,54.9408,48
.4692,48.0375,50.2621,66.2382,57.1459,44.2686,53.3578,60.1617,69.0782,
57.9543,41.1887,61.6895,56.9442,64.7135,46.7303,55.0103,49.7727,52.725
1,70.0936,61.0781,37.7842,74.5408,54.0033,53.9978,46.9408,51.6053,68.5
506,55.6076,60.6455,61.4612,51.8165,55.3066,56.0769,72.6906,62.7462,48
.2955,58.2015,57.8313,65.5088,57.4774,77.4424,51.6938,59.7658,86.8949,
59.8112,51.1317,53.0629,45.2534,47.141,51.0053,56.1231,61.6508,63.3491
,43.6161,62.4182,69.0119,47.2942,58.7787,60.1584,58.1066,63.5981,58.60
36,59.0922,60.3486,53.5367,48.7594,59.7507,56.4324,65.9085,68.1473,52.
99]},{"error_x":{"color":"rgba(255,0,0,1)"},"error_y":
{"color":"rgba(255,0,0,1)"},"line":
{"color":"rgba(255,0,0,1)"},"marker":
{"color":"rgba(255,0,0,1)","line":
{"color":"rgba(255,0,0,1)"},"size":5},"mode":"markers","name":"TRUE","
textfont":{"color":"rgba(255,0,0,1)"},"type":"scatter3d","x":
[33.4722,30.5224,31.0351,40.3897,38.5563,28.6931,41.6121,35.8518,37.68
99,38.1645,43.0961,36.7826,36.3355,35.5195,41.2931,44.6121,45.3881,45.
7488,34.6181,36.8566,50.2747,39.1551,44.2909,38.8378,51.3774,52.7615,3
8.7614,32.4582,42.4215,40.1199,44.2581,57.824,41.8939,43.5789,41.0187,
46.4921,59.9832,42.6872,53.3831,47.5098,36.7429,49.2309,57.8858,47.778
3,44.2629,48.0348,35.826,49.995,51.2007,58.8531,49.4258,59.4081,42.834
7,60.1478,53.4391,60.7586,55.7491,39.8385,46.2058,53.9099,47.3499,47.5
785,41.9022,60.0089,48.0764,52.8205,54.719,63.7922,64.9018,56.7667,59.
5873,52.0278,48.8769,62.9156,48.7716,59.3055,51.4583,57.4772,51.4222,5
2.0457,58.4327,56.1067,51.6278,62.4744,66.005,59.1678,63.363,49.595,49
.8242,57.3588,56.9473,54.879,46.7982,51.7775,64.2852,53.6613,63.3596,5
```

```
3.994,55.4032,61.5619,59.9409,56.8559,55.8483,52.1276,50.2508,52.0327,
55.7987,41.7653,60.3359,60.7845,55.241,50.5235,50.18,52.3394,52.7672,5
8.6807,49.1904,56.542,54.4102,46.6836,60.3839,50.267,51.4381,52.4387,5
7.9974,41.9411,37.5136,41.8028,61.3363,57.1775,36.9694,55.0686,40.3618
,60.7126,42.5234,42.6378,47.8747,43.2149,39.0551,61.3136,40.3276,42.06
7,47.3393,46.4722,45.3499,43.5577,41.2686,64.273,29.5159,56.2661,54.06
84,42.4489,32.6011,36.9053,44.1118,39.2765,32.9833,38.0554,36.695,43.4
131,31.4814,39.1282,38.781,37.5145,41.2749,35.4744,43.5454,35.9076,24.
8532,54.6684,39.552],"y":
[71.6574,75.4255,70.0617,83.1694,80.2085,78.2843,78.1755,74.8859,80.13
89,65.4001,94.435,70.437,74.1329,62.4299,67.7125,70.5055,72.0003,65.64
57,67.6142,70.3654,68.4549,82.7162,81.4951,73.4288,75.6099,74.7921,91.
6861,87.8931,70.2081,83.0095,93.8459,70.4123,68.5294,72.6758,71.2913,7
2.5141,68.8938,77.0048,90.3319,78.8625,79.9698,67.2074,67.5082,68.9275
,68.2148,69.0254,62.3326,72.2615,79.0817,65.5814,63.9919,75.5835,80.06
62,77.6853,67.2989,75.2549,77.1007,77.3953,81.9997,85.3571,71.6985,69.
001,80.9495,93.4858,74.2012,95.3153,85.3491,72.3242,85.5041,68.8442,68
.9777,71.8133,84.7832,78.2383,95.1017,71.6552,70.5052,70.0866,72.3128,
84.2972,73.5565,70.5944,85.4198,86.3172,62.565,78.708,81.0006,92.647,8
2.6746,65.0434,76.7423,75.2027,70.798,79.7094,69.2338,67.4436,82.1369,
74.1934,82.2131,77.3775,69.984,77.8533,74.9686,62.2553,64.9065,66.7343
,82.0219,65.368,79.7371,70.812,79.0775,78.4978,80.6062,83.8644,77.2065
,82.7272,68.7227,69.8984,84.4692,85.8625,64.9655,74.7736,81.7632,80.56
29,67.7873,66.0876,75.4806,69.0585,71.3454,79.2492,75.4465,69.7421,80.
3272,85.516,82.1738,72.0061,84.5747,72.694,73.1938,81.1113,65.1525,68.
8208,87.8832,68.9851,65.7255,59.6531,84.8964,83.2075,69.9612,74.1718,6
6.7076,66.4055,70.8797,73.458,71.5688,72.7928,76.3381,63.4233,83.846,6
8.4981,69.0705,72.8758,79.8543,65.4041,78.5708,81.5571,75.4286,70.029,
71.1823,61.4846,73.369],"z":
[38.1852,44.9031,39.0267,42.7797,41.6522,49.5911,36.5633,39.034,42.449
,27.2356,51.3389,33.6544,37.7974,26.9104,26.4194,25.8934,26.6123,19.89
7,32.9961,33.5088,18.1802,43.5611,37.2041,34.591,24.2324,22.0306,52.92
47,55.435,27.7866,42.8896,49.5877,12.5883,26.6355,29.0969,30.2726,26.0
219,8.9106,34.3177,36.9488,31.3527,43.2269,17.9765,9.6224,21.1493,23.9
519,20.9906,26.5066,22.2665,27.8809,6.7283,14.5661,16.1753,37.2315,17.
5375,13.8598,14.4963,21.3516,37.5568,35.7939,31.4473,24.3486,21.4225,3
9.0472,33.477,26.1248,42.4948,30.6301,8.5321,20.6023,12.0775,9.3904,19
.7856,35.9062,15.3227,46.3301,12.3497,19.047,12.6094,20.8906,32.2515,1
5.1238,14.4878,33.792,23.8428,-
3.4399,19.5403,17.6376,43.052,32.8504,7.6846,19.7951,20.3237,23.9998,2
7.9319,4.9486,13.7823,18.7774,20.1994,26.8099,15.8157,10.0431,20.9974,
19.1203,10.1277,14.6558,14.7016,26.2232,23.6026,19.4012,10.0275,23.836
6,27.9742,30.4262,31.525,24.4394,24.0465,19.5322,13.3564,30.059,39.178
9,4.5816,24.5066,30.3252,28.1242,9.7899,24.1465,37.967,27.2557,10.009,
22.0718,38.4771,14.6736,39.9654,24.8034,39.6504,29.3684,36.7,29.4791,3
4.1388,19.7976,24.825,26.7538,40.5439,22.5128,20.3756,16.0954,43.6278,
18.9345,40.4453,17.9057,12.6392,23.9566,38.2785,36.5526,27.4571,33.516
3,43.3548,25.3678,47.1511,25.085,37.5891,33.7476,41.0733,27.8896,37.29
59,46.0827,31.8832,34.1213,46.3291,6.8162,33.817]}],"highlight":
```

{"debounce":0,"dynamic":false,"on":"plotly_click","opacityDim":0.2,"persistent":false,"selected":{"opacity":1},"selectize":false},"layout":{"hovermode":"closest","margin":{"b":40,"l":60,"r":10,"t":25},"scene":{"xaxis":{"title":"Production"},"yaxis":{"title":"Demand"},"zaxis":{"title":"Shortage"}},"showlegend":true,"title":"Production vs Demand vs Shortage"},"shinyEvents":["plotly_hover","plotly_click","plotly_selected","plotly_relayout","plotly_brushed","plotly_brushing","plotly_clickannotation","plotly_doubleclick","plotly_deselect","plotly_afterplot","plotly_sunburstclick"],"source":"A","visdat":{"58f47eb425d0":["function () ","plotlyVisDat"]}}

As we can see the from the "Production vs Demand vs Shortage" plot, the system operates in a failure-prone region of the state space of the region (all have positive shortage). This suggests the system lacks a "safe operating zone" - failures can occur under various combinations of production and demand.

Key Insights

The analysis confirms our earlier calculation: with production averaging only 46% of demand, the system is fundamentally undersized. The visualizations reveal this isn't just an average problem - even on the best production days, the system rarely generates surplus energy. This explains why P(at least 10 failures) = 1.0 with such high confidence.

Recommendation

For the system to operate in a "safe operating zone" where demands are met they, could:

- Somehow improve the production by increasing it at $\sim 2.2\,x$.
  - This could be done adding more panels or
  - cleaning the panel regularly to maximize the solar energy.

  Note that increasing the capacity, with batteries for example, would achieve minimal improvement since there really is no excess energy to store.
- Lower the demand. If there's anything the could be infer to this whole report, it is that the whole system isn't ready for production at large (even just among remote villages).
  - Limit the allowed electronics on the households, like limit it to just fan during the day and light (and fan) during night.
  - If we do that, the system could now very well benefit with battery since we will need to utilize the time of the day where solar intensity is at its highest and store the energy gathered somewhere.